

mini-swe-agent + SWE-bench Lite 多 Case 评测与失败恢复：阶段性学习与实验报告

截至 2026-09-04 的批量评测、trajectory 解析、Visible Counterexample Guard、真实实验和后续 Recovery 路线图。

当前节点已经形成可重复的批量 Agent Eval 链路

截至 2026-09-04，项目已从单 case 手工流程推进到：**串行批量运行、逐 case 隔离、trajectory/patch/prediction 归档、结构化解析、SWE-bench 4.0.4 正式判分、失败分析，以及可配置的 Visible Counterexample Guard**。10 个定向 baseline 中，人工复核后为 **3 Resolved / 4 有效 Unresolved / 3 Error**；7 个有效判分 case 的成功率为 **42.9%**。这不是完整 SWE-bench Lite 榜单分数，而是用于工程验证和失败样本采集的选择性样本。

Guard v1 证明了“提交前拦截 Agent 自己已经看见但尚未解决的反例”可以工作，但也暴露了 piped pytest evidence 无法关闭导致无限调用的风险。Guard v2 已增加 normalized pytest target closure 和有界终止：单元测试 **48 passed / 1 skipped**，两个真实重跑均在 3 次 Submit block 后安全结束，不再无限调用模型。当前最优下一步不是继续扩大样本，而是实现一轮受限、可审计的 Recovery，并用固定小样本做 paired rerun。

Source: baseline10_20260903_isolated_v1 reviewed summary

Table: baseline

10-case baseline 同时暴露了修复失败与评测基础设施风险

3 个 Resolved 证明批量 Agent 生成与 Harness 判分链路可以闭环；4 个有效 Unresolved 提供了真实 failure analysis 样本；另外 3 个 Error 不能当成模型修复失败，其中两个缺 Docker image，一个在评测环境加载阶段失败。报告后续所有成功率和失败策略均优先使用 7 个有效判分 case，而不是把基础设施错误混入模型质量。

Source: baseline10_20260903_isolated_v1 reviewed summary

Table: baseline

10-case baseline 的复核结果

Source: baseline10_20260903_isolated_v1 reviewed summary

Table: baseline

复核状态	Case 数	口径
Resolved	3	Harness 有效判分且通过
Unresolved	4	Harness 有效判分但未通过
Error	3	镜像或评测基础设施问题

Baseline 10-case 明细

Source: baseline10_20260903_isolated_v1 reviewed summary

Table: baseline

#	Instance	复核状态	Steps	Cost USD	Runtime s	失败类型
3	marshmallow-code__marshmallow-1343	Resolved	40	\$0.14	301.13	-
2	marshmallow-code__marshmallow-1359	Unresolved	21	\$0.06	107.36	target_tests_failed
4	pvlb__pvlb-python-1072	Error	15	\$0.01	109.14	evaluation_environment
5	pvlb__pvlb-python-1606	Error	—	—	—	image_unavailable
10	pydicom__pydicom-1256	Resolved	18	\$0.04	69.22	-
9	pydicom__pydicom-1413	Unresolved	44	\$0.18	607.75	regression_tests_failed
8	pydicom__pydicom-1694	Resolved	25	\$0.04	100.57	-
6	pylint-dev__astroid-1196	Error	—	—	—	image_unavailable
7	pylint-dev__astroid-1866	Unresolved	17	\$0.01	76.64	target_tests_failed
1	sqlfluff__sqlfluff-1517	Unresolved	74	\$0.39	610.8	target_tests_failed

四类 Agent 失败应由不同 Recovery 触发器处理

1. 可见反例未闭环

Agent 已通过自己运行的复现或测试看见明确失败，却仍尝试 Submit。marshmallow-1359 是最直接样本：baseline 的目标测试失败，而 Guard v1 在提交前拦截后促成复测并最终 Resolved。

2. 回归覆盖不足

pydicom-1413 修复了 3 个 FAIL_TO_PASS，却使 2 个原 PASS_TO_PASS 失败。这不是“没有修到目标”，而是局部验证通过后没有充分覆盖相邻行为。Recovery 需要基于修改文件和已发现测试范围扩大验证，不能读取隐藏测试。

3. 长轨迹低进展或 stuck

sqlfluff-1517 使用 74 steps、约 \$0.392 和 611 秒，最终目标测试仍失败。典型信号包括重复搜索、重复修改、连续测试没有状态改善，以及在证据没有变化时继续消耗调用。该类问题不应由 Counterexample Guard 无限延长，而应由独立的 low-progress detector 触发一次计划重置，随后有界终止。

4. 目标修复不完整或方向偏差

pylint-astroid-1866 和 baseline 的 marshmallow-1359 都保留 FAIL_TO_PASS 失败，但没有回归失败。这类情况更适合重新检查任务约束、最小复现和语义边界，而不是先扩大测试。

基础设施错误应单列：image unavailable、依赖/评测环境不兼容、无合法 prediction。它们由 preflight 和评测编排处理，不进入 Agent Recovery。

Source: baseline10_20260903_isolated_v1 reviewed summary

Table: baseline

Guard v1 证明机制有效，但 piped pytest closure 曾造成不可终止路径

四个 A/B case 中，3 个生成合法 prediction 并经 Harness 全部 Resolved。marshmallow-1359 从 baseline Unresolved 变为 Resolved；两个原 Resolved 对照保持 Resolved。与此同时，pydicom-1256 的三条 piped pytest evidence 因 v1 只接受相同 command 成功复测而无法关闭，造成 8 次 Submit block、74 steps 和 \$0.398 后人工安全中断。由于模型采样存在随机性，跨运行 steps/cost 差值不能全部解释为 Guard 因果开销；直接可信的是 trigger、block、evidence 和 Harness 状态。

Source: guard_ab4_pro_retry_20260904_3QIIB2 comparison

File: experiments/guard_ab4_pro_retry_20260904_3QIIB2/comparison.json

Visible Counterexample Guard v1 真实 A/B

Source: guard_ab4_pro_retry_20260904_3QIIB2 comparison

File: experiments/guard_ab4_pro_retry_20260904_3QIIB2/comparison.json

Instance	Baseline	Guard/Harness	Steps B to G	G cost	Blocks	Open evidence
marshmallow-code__marshmallow-1343	Resolved	Resolved	40 to 36	\$0.03	1	0
marshmallow-code__marshmallow-1359	Unresolved	Resolved	21 to 30	\$0.02	1	0
pydicom__pydicom-1256	Resolved	Error	18 to 74	\$0.4	8	3
pydicom__pydicom-1694	Resolved	Resolved	25 to 22	\$0.01	0	0

Guard v2 已修复 target closure 与有限终止，但尚未证明提升总体成功率

实现新增：pytest verification_target 规范化；piped failure 标记 closure_requires_reliable_rerun；相同 target 的可靠非 pipe passed 可以关闭 evidence；不同 target、timeout、no_tests、unknown 不可关闭；无法可靠解析的命令继续采用精确 command 复测。默认最多 3 次 Submit block，首次触发后最多 24 个 recovery steps。

真实重跑中，pydicom-1256 从 v1 的 74 steps / 8 blocks 降到 29 steps / 3 blocks，并以 GuardUnresolvedEvidence 干净结束；marshmallow-1359 同样在第 3 次 block 终止。两者都没有有效补丁，因此没有运行 Harness，也不能写成 Unresolved。normalized target 跨命令 closure 已由单元测试和使用旧轨迹命令的离线 dry-run 验证，但这次随机在线轨迹没有形成相同现场序列。

Source: guard_v2_two_20260904_dHWhVh results

File: experiments/guard_v2_two_20260904_dHWhVh/results.json

Guard v2 有界终止重跑

Source: guard_v2_two_20260904_dHWhVh results

File: experiments/guard_v2_two_20260904_dHWhVh/results.json

Instance	Steps	Cost	Runtime s	Blocks / open	终止原因
marshmallow-code__marshmallow-1359	54	\$0.24	868.76	3 / 3	max total blocks
pydicom__pydicom-1256	29	\$0.16	681.99	3 / 3	max evidence-group blocks

口径、范围与证据边界

- **Agent**: 当前使用 mini-swe-agent 2.4.6 的默认 SWE-bench agent 流程，模型为 deepseek/deepseek-v4-pro。
- **生成与判分分离**: mini-swe-agent 生成 trajectory 和 patch；传统 SWE-Bench_Lite 使用固定 swebench==4.0.4、python -m swebench.harness.run_evaluation 判分。
- **Submitted 不等于 Resolved**: 只有 Harness 完成 FAIL_TO_PASS/PASS_TO_PASS 后才标记 Resolved/Unresolved。
- **Steps/API calls**: trajectory 中可观察 assistant 模型响应数和上游 api_calls；不推断底层 HTTP 重试。
- **Cost**: 上游模型统计报告的 USD 估算。
- **Runtime**: batch runner 通过单调时钟测得的单 case Agent 子进程耗时，不含 image pull、数据准备和 Harness。
- **安全边界**: Agent 运行时不读取 gold patch、参考修复提交、隐藏测试期望或 Harness 结果。Failure analysis 可以读取已经完成实验的 Harness 日志，但这些结果不回灌给 Agent。

Source: mini-swe-agent v2.4.6 implementation and tests

File: src/minisweagent/run/benchmarks

当前实现是独立编排与 policy 层，没有重写 Agent 核心

batch_runner

按输入顺序串行调用现有 SWE-bench CLI；每 case 独立目录；单 case 失败不阻断后续；保存 trajectory、prediction、patch、parsed metrics、runner log 和 run metadata；逐 case 原子更新 summary.json，并聚合合法 predictions.jsonl。

trajectory_parser

纯解析现有 trajectory，提取 steps、API calls、cost、runtime、bash/tool calls、测试状态、修改文件、exit status 和 Guard 元数据；不执行轨迹命令，不把轨迹路径当作本机读写目标。

test_status 与 Visible Counterexample Guard

至少区分 passed / failed / timeout / no_tests / unknown；避免把 pytest | head/tail 的外层 0 当成测试成功。Guard 只使用 Agent 自己可见的命令和输出形成 evidence，在 Submit 前检查 closure；到界后安全终止。

验证

当前相关测试为 48 passed / 1 skipped。skip 是未提供外部历史 trajectory fixture；没有为本轮安装或升级依赖。

Source: mini-swe-agent v2.4.6 implementation and tests

当前结论的限制

1. 10 个 case 是面向镜像可用性和失败样本采集的定向选择，不可外推为 SWE-bench Lite 总体成功率。
2. 每个 treatment 只有单次随机轨迹；模型采样差异会显著影响 steps、cost 和最终补丁，不能把全部差值归因于 Guard。
3. Guard v2 真实重跑直接验证了有界终止，但 target-based cross-command closure 只在单元测试和离线 dry-run 中被命中。
4. 当前 test parser 只保守识别常见 pytest/unittest/tox/manage.py 入口；复杂 shell、动态脚本和自定义测试框架可能仍为 unknown。
5. 两个 image unavailable 和一个 evaluation environment error 说明扩大样本前仍需更强的 preflight 与镜像清单。
6. 本阶段没有实现通用 stuck recovery、checkpoint rollback、自动 Harness 评测编排或并发调度。

后续优先实现两类有界 Recovery

Recovery 1: Evidence-directed verification loop（优先）

当 Submit 被 Guard 阻止时，向 Agent 注入一份短状态摘要：未关闭 evidence、对应 verification target、最近一次失败摘要、当前 diff 和允许的剩余 recovery rounds。要求 Agent 先选择一种可审计动作：相同 target 的可靠复测、基于可见证据说明与任务无关、或提供环境问题证据。每个 evidence group 最多 1-2 个 recovery rounds；失败后保持 GuardUnresolvedEvidence 终止。

在提交前增加分层验证：第 1 层运行最小复现；第 2 层运行与修改文件直接相关的测试模块；第 3 层在预算允许时运行更宽的本地回归集。测试集合只能从仓库结构、已读代码、任务描述和 Agent 自己发现的测试得到，不能读取隐藏 Harness 预期。若较宽测试出现新 regression，优先回到最近一次通过 checkpoint 或缩小补丁。

Recovery 2: Low-progress / stuck reset（第二优先）

用滚动窗口跟踪 Agent 可见的进展事件：新增有效证据、测试状态改善、diff 变化、修改文件变化、或明确缩小问题范围。若连续 N 个模型步骤没有任何进展，且出现重复搜索、重复命令、重复修改或连续测试无改善，则触发一次 plan reset：总结已尝试路径、保留/否定的假设、当前最小失败和下一条不同策略。每 case 最多一次 reset；再无进展则安全终止，不能把 stuck detector 变成新的无限循环。

暂缓项目

并发、数据库、Web UI、跨进程 Recovery、自动拉取缺失镜像和复杂 shell 等价推断继续后置。先用固定 4-6 case 做 paired rerun，观察 Resolved、false block、额外 steps/cost、终止率和 regression 数。

建议的下一轮实验与验收标准

1. 固定 marshmallow-1359、pydicom-1256、pydicom-1413、sqlfluff-1517，再加入 1-2 个稳定 Resolved 对照。
2. 为每个 case 固定数据、镜像 ID、模型、配置、网络隔离和预算；分别运行 Guard-only 与 Guard+Recovery。
3. 每个 case 至少做 2-3 次重复，避免把随机轨迹当作单次因果证据。
4. 主指标：Harness Resolved；护栏：false block、Guard termination、regression、额外 steps/cost、有效 prediction 比例。

5. 通过门槛：没有无限调用；原稳定 Resolved 对照不出现系统性 regression；Unresolved 至少出现可解释的改善或更低成本的安全终止。
6. 只有在小样本满足门槛后，再扩大到 20-30 个镜像已就绪 case。

项目复现与学习清单

- 代码仓库：jlz61/mini-swe-agent，基于上游 v2.4.6。
- Agent 环境：conda activate mini-swe-agent。
- Harness 环境：conda activate swebench-eval-v4，固定 SWE-bench 4.0.4。
- 服务器源码：/home/jiangpy/projects/mini-swe-agent。
- 数据：/data1/jiangpy/datasets/SWE-Bench_Lite，离线使用。
- 批量实验：/data1/jiangpy/mini-swe-agent-runs。
- Harness 输出：/data1/jiangpy/swebench-evals 及各实验的 evaluation/logs/run_evaluation。
- 共享 DockerRootDir：/scratch2/docker/docker；禁止清理其他用户资源。
- 全局模型配置位于用户配置目录，不应提交到 Git。
- 继续实验前始终检查 git status，不要提交 .env、trajectory、prediction、日志、Docker 数据或 __pycache__。

仍需回答的问题

- normalized target 应否支持 -k/-m 等选择表达式，如何保持保守且可关闭？
- Python traceback evidence 是否需要可声明的 semantic target，而不只依赖 exact command？
- 分层回归测试如何从修改文件稳定映射到测试集合，同时控制成本？
- stuck detector 的窗口与阈值在不同仓库规模下是否需要自适应？
- 多次随机重复后，Guard/Recovery 对成功率、成本和 false block 的真实影响是多少？